

Credenciales guardadas en Windows

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Muchas escaladas en Windows no son un exploit, sino encontrar una credencial reutilizable. Windows guarda passwords en ficheros de despliegue (unattend.xml, sysprep), en el gestor de credenciales (cmdkey), en el registro (Winlogon autologon, PuTTY), y en backups del SAM/SYSTEM que se pueden volcar. Cada credencial alimenta cracking o reuse directo.

Los administradores automatizan despliegues y dejan credenciales en claro o cifradas de forma reversible. Y el propio Windows cachea secretos para comodidad. Buscar sistemáticamente es de lo mas rentable, igual que en Linux.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeracion (enum-privesc-windows), en paralelo a los vectores de servicio/token. WinPEAS busca la mayoría automáticamente.

Cómo identificarla

- Ficheros unattend.xml / sysprep.xml / Autounattend.xml en el disco.
- cmdkey /list muestra credenciales cacheadas (runas /savecred).
- Autologon en el registro con DefaultPassword, o hives SAM/SYSTEM accesibles.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

findstr /si password sobre configs y reg query del autologon en Winlogon.

Evidencia esperada: Una password en claro en un fichero de config, o DefaultPassword en el registro.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v DefaultPassword
```

Objetivo y contexto: Si hay autologon configurado, la password del usuario queda en claro en el registro. Es un clasico de rooms Windows.

Resultado esperado: DefaultUserName y DefaultPassword en claro si existe. Esa credencial -> pruebala con runas o para RDP/WinRM; a menudo es de un usuario mas privilegiado.

Capa 2: prueba documentada

```
findstr /si password *.xml *.ini *.txt *.config
```

Objetivo y contexto: Peina ficheros de configuracion y despliegue buscando 'password'. /s recursivo, /i sin distinguir mayusculas. Barato y muy rentable.

Resultado esperado: Lineas con credenciales y su fichero. unattend.xml suele traer la password del admin local (a veces en base64: decodificala).

Anatomía de los payloads

La primera prueba es `reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v DefaultPassword`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una password en claro en un fichero de config, o DefaultPassword en el registro.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Busca ficheros de despliegue con credenciales en claro o base64.
- Lista credenciales cacheadas con cmdkey; usa runas /savecred si hay.
- Revisa el registro (autologon, claves de apps) y volca SAM/SYSTEM si puedes.
- Cada credencial -> pruebala (reuse) o crackeala (cracking); un hash NTLM sirve para ntlm-pth.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay ficheros de despliegue (unattend.xml), autologon ni credenciales cacheadas visibles.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Windows deja credenciales por todas partes: ficheros de despliegue, gestor de credenciales, registro y backups del SAM. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (stored-credentials-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.